# Value Expressions Fundamentals

**Value Expressions Fundamentals**



Value Expressions are a combination of Spring Expression Language (SpEL) and Property Placeholder Resolution. They combine powerful evaluation of programmatic expressions with the simplicity to resort to property-placeholder resolution to obtain values from the `Environment` such as configuration properties.

Expressions are expected to be defined by a trusted input such as an annotation value and not to be determined from user input.

## Scope

Value Expressions are used in contexts across annotations. Spring Data offers Value Expression evaluation in two main contexts:

- **Mapping Model Annotations**: such as `@Document`, `@Field`, `@Value` and other annotations in Spring Data modules that ship with their own mapping models respective Entity Readers such as MongoDB, Elasticsearch, Cassandra, Neo4j. Modules that build on libraries providing their own mapping models (JPA, LDAP) do not support Value Expressions in mapping annotations.

  The following code demonstrates how to use expressions in the context of mapping model annotations.

  *Example 1. `@Document` Annotation Usage*

  ```java
  @Document("orders-#{tenantService.getOrderCollection()}-${tenant-config.suffix}")
  class Order {
  ```

```
  // …
}
```

- **Repository Query Methods**: primarily through `@Query`.

  The following code demonstrates how to use expressions in the context of repository query methods.

  *Example 2. @Query Annotation Usage*

```java
class OrderRepository extends Repository<Order, String> {

  @Query("select u from User u where u.tenant = ?${spring.application.name:unknown}
and u.firstname like %?#{escape([0])}% escape ?#{escapeCharacter()}")
  List<Order> findContainingEscaped(String namePart);
}
```

NOTE

Consult your module's documentation to determine the actual parameter by-name/by-index binding syntax. Typically, expressions are prefixed with `:#{…}` / `:${…}` or `?#{…}` / `?${…}`.

## Expression Syntax

Value Expressions can be defined from a sole SpEL Expression, a Property Placeholder or a composite expression mixing various expressions including literals.

*Example 3. Expression Examples*

```
#{tenantService.getOrderCollection()}                          1
#{(1+1) + '-hello-world'}                                      2
${tenant-config.suffix}                                        3
orders-${tenant-config.suffix}                                 4
#{tenantService.getOrderCollection()}-${tenant-config.suffix}  5
```

1. Value Expression using a single SpEL Expression.
2. Value Expression using a static SpEL Expression evaluating to `2-hello-world`.
3. Value Expression using a single Property Placeholder.
4. Composite expression comprised of the literal `orders-` and the Property Placeholder `${tenant-config.suffix}`.
5. Composite expression using SpEL, Property Placeholders and literals.

NOTE

NOTE

Using value expressions introduces a lot of flexibility to your code. Doing so requires evaluation of the expression on each usage and, therefore, value expression evaluation has an impact on the performance profile.

Spring Expression Language (SpEL) and Property Placeholder Resolution explain the syntax and capabilities of SpEL and Property Placeholders in detail.

## Parsing and Evaluation

Value Expressions are parsed by the `ValueExpressionParser` API. Instances of `ValueExpression` are thread-safe and can be cached for later use to avoid repeated parsing.

The following example shows the Value Expression API usage:

*Parsing and Evaluation*

Java | **Kotlin**

JAVA

```
ValueParserConfiguration configuration = SpelExpressionParser::new;
ValueEvaluationContext context = ValueEvaluationContext.of(environment,
evaluationContext);

ValueExpressionParser parser = ValueExpressionParser.create(configuration);
ValueExpression expression = parser.parse("Hello, World");
Object result = expression.evaluate(context);
```

## SpEL Expressions

SpEL Expressions follow the Template style where the expression is expected to be enclosed within the `#{…}` format. Expressions are evaluated using an `EvaluationContext` that is provided by `EvaluationContextProvider`. The context itself is a powerful `StandardEvaluationContext` allowing a wide range of operations, access to static types and context extensions.

NOTE

Make sure to parse and evaluate only expressions from trusted sources such as annotations. Accepting user-provided expressions can create an entry path to exploit the application context and your system resulting in a potential security vulnerability.

### Extending the Evaluation Context

`EvaluationContextProvider` and its reactive variant `ReactiveEvaluationContextProvider` provide access to an `EvaluationContext`. `ExtensionAwareEvaluationContextProvider` and its reactive vari-

ant `ReactiveExtensionAwareEvaluationContextProvider` are default implementations that determine context extensions from an application context, specifically `ListableBeanFactory`.

Extensions implement either `EvaluationContextExtension` or `ReactiveEvaluationContextExtension` to provide extension support to hydrate `EvaluationContext`. That are a root object, properties and functions (top-level methods).

The following example shows a context extension that provides a root object, properties, functions and an aliased function.

*Implementing a `EvaluationContextExtension`*

Java | **Kotlin**

JAVA

```
@Component
public class MyExtension implements EvaluationContextExtension {

    @Override
    public String getExtensionId() {
        return "my-extension";
    }

    @Override
    public Object getRootObject() {
        return new CustomExtensionRootObject();
    }

    @Override
    public Map<String, Object> getProperties() {

        Map<String, Object> properties = new HashMap<>();

        properties.put("key", "Hello");

        return properties;
    }

    @Override
    public Map<String, Function> getFunctions() {

        Map<String, Function> functions = new HashMap<>();

        try {
            functions.put("aliasedMethod", new
Function(getClass().getMethod("extensionMethod")));
            return functions;
        } catch (Exception o_O) {
```

```
            throw new RuntimeException(o_O);
        }
    }

    public static String extensionMethod() {
        return "Hello World";
    }

    public static int add(int i1, int i2) {
        return i1 + i2;
    }

}

public class CustomExtensionRootObject {

    public boolean rootObjectInstanceMethod() {
        return true;
    }

}
```

Once the above shown extension is registered, you can use its exported methods, properties and root object to evaluate SpEL expressions:

*Example 4. Expression Evaluation Examples*

```
#{add(1, 2)}                                    1
#{extensionMethod()}                            2
#{aliasedMethod()}                              3
#{key}                                          4
#{rootObjectInstanceMethod()}                   5
```

1. Invoke the method `add` declared by `MyExtension` resulting in `3` as the method adds both numeric parameters and returns the sum.
2. Invoke the method `extensionMethod` declared by `MyExtension` resulting in `Hello World`.
3. Invoke the method `aliasedMethod`. The method is exposed as function and redirects into the method `extensionMethod` declared by `MyExtension` resulting in `Hello World`.
4. Evaluate the `key` property resulting in `Hello`.
5. Invoke the method `rootObjectInstanceMethod` on the root object instance `CustomExtensionRootObject`.

You can find real-life context extensions at `SecurityEvaluationContextExtension`.

## Property Placeholders

Property placeholders following the form `${…}` refer to properties provided typically by a `PropertySource` through `Environment`. Properties are useful to resolve against system properties, application configuration files, environment configuration or property sources contributed by secret management systems. You can find more details on the property placeholders in Spring Framework's documentation on `@Value` usage.